

Disentangling Writer and Character Styles for Handwriting Generation

Gang Dai^{1*}, Yifan Zhang^{2*}, Qingfeng Wang¹, Qing Du¹, Zhuliang Yu¹,
 Zhuoman Liu³, Shuangping Huang^{1,4†}

¹South China University of Technology, ²National University of Singapore,

³The Hong Kong Polytechnic University, ⁴Pazhou Laboratory

eedaigang@mail.scut.edu.cn, yifan.zhang@u.nus.edu, eehsp@scut.edu.cn

Abstract

Training machines to synthesize diverse handwritings is an intriguing task. Recently, RNN-based methods have been proposed to generate stylized online Chinese characters. However, these methods mainly focus on capturing a person’s overall writing style, neglecting subtle style inconsistencies between characters written by the same person. For example, while a person’s handwriting typically exhibits general uniformity (e.g., glyph slant and aspect ratios), there are still small style variations in finer details (e.g., stroke length and curvature) of characters. In light of this, we propose to disentangle the style representations at both writer and character levels from individual handwritings to synthesize realistic stylized online handwritten characters. Specifically, we present the style-disentangled Transformer (SDT), which employs two complementary contrastive objectives to extract the style commonalities of reference samples and capture the detailed style patterns of each sample, respectively. Extensive experiments on various language scripts demonstrate the effectiveness of SDT. Notably, our empirical findings reveal that the two learned style representations provide information at different frequency magnitudes, underscoring the importance of separate style extraction. Our source code is public at: <https://github.com/dailenson/SDT>.

1. Introduction

As the oldest writing system, Chinese characters are widely used across Asian countries. When compared with Latin scripts, Chinese characters encompass an exceptionally vast lexicon (87,887 characters in GB18030-2022 charset) and have intricate structures composed of multiple strokes. Recently, the intriguing task of generating Chinese characters has garnered significant attention [10, 23, 32]. A promising approach to synthesising realistic handwritings is

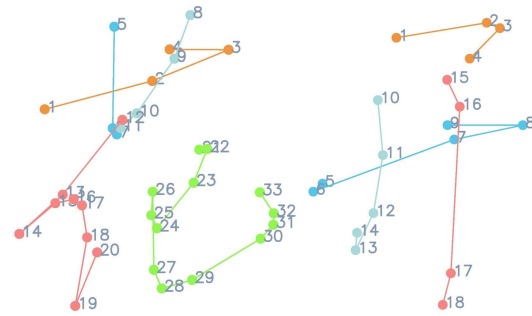


Figure 1. Illustration of two online handwritten Chinese characters, with each color representing a stroke. The increasing numbers indicate the writing order from the start to the end.

to progressively generate online characters (*i.e.*, the handwriting trajectory in a sequential format) [40]. As shown in Figure 1, online characters convey richer information (*e.g.*, the order of writing) and thus pave the way for various applications, including writing robots [39].

Our goal is to automatically generate online Chinese handwritings that not only correspond to specific textual content, but also emulate the calligraphic style of a given exemplar writer (*e.g.*, glyph slant, shape, stroke length, and curvature). This task thus holds potential for a wide range of applications, such as font design and calligraphy education. A popular solution [18] is to extract style information from the provided stylized samples and merge it with the content reference. DeepImitator [44] concatenates the style vector obtained from a CNN encoder with a character embedding, which is then fed into the RNN decoder to generate stylized online characters. WriteLikeYou [30] adopts the large-margin softmax loss [36] to promote discriminative learning of style features. However, these methods mainly focus on the overall writing style, thus overlooking the detailed style inconsistencies (*e.g.*, the highlighted regions in Figure 2) between characters produced by the same writer.

The observations mentioned above inspire us to disentangle style representations at the writer and character levels from the stylized handwritings. However, accurately cap-

* Authors contributed equally.

† Corresponding author

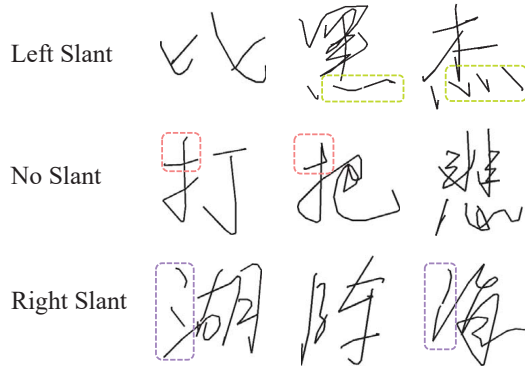


Figure 2. Handwritten character samples from three unique writers, with each row containing characters by the same person. Despite sharing similar overall handwriting styles (e.g., glyph slant), subtle style differences (e.g., stroke length, location, and curvature) can still be observed among them.

turing these two styles is a challenging task. To address this, we propose a style-disentangled Transformer (SDT) equipped with a dual-head style encoder. Specifically, we employ the contrastive learning framework [13] to guide each head in concentrating on writer-wise and character-wise styles, respectively. For the overall writer-wise style, we treat characters from the same writer as positive instances, and characters from different writers as negatives. This enables the encoder to learn the style commonalities among characters written by the same writer. Regarding the detailed character-wise style, we independently sample positive pairs within a character, and sample negative samples from other characters, as illustrated in Figure 3. Aggregating positive views of a character encourages the encoder to focus on the intricate character style patterns.

In addition, we introduce a content encoder for SDT to learn a textual feature with a global context. The two style representations, along with the textual feature, are then fed into a decoder that progressively generates online characters. Given that the output characters are in sequential form, we employ Transformer [35], a powerful sequence modeling architecture, as our backbone.

To extend SDT for generating offline Chinese handwritings (i.e., character images with stroke-width, e.g., Figures 3-7 in Appendix), we further propose an offline-to-offline generation framework. We first use SDT to generate online characters with significant shape changes, and then decorate them with stroke width, ink-blot effects, etc. This enables us to generate authentic offline handwritings. For more details, please refer to Appendix A.4.

We summarize our contributions in three key aspects:

- We are the first to disentangle two style representations (i.e., writer-wise and character-wise) for enhancing Chinese handwriting generation. Our findings show that the former focuses more on low-frequency information, while the latter captures higher frequencies.

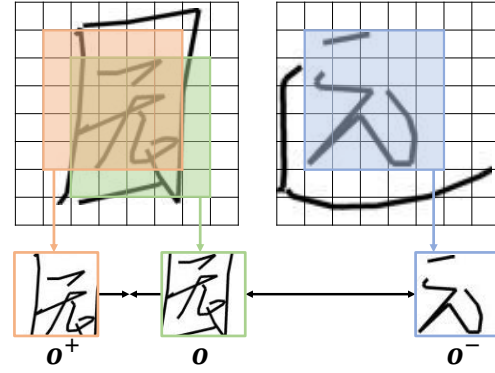


Figure 3. In this two-character example, we independently sample the positive pair, i.e., o and o^+ , within the first character, while the negative o^- is sampled from another character. Our sampling strategy randomly selects a small subset of patches, following a uniform distribution.

- We introduce a novel online character generation method, i.e., SDT. Extensive experiments on handwriting datasets in Chinese, English, Japanese, and Indic scripts demonstrate its effectiveness and superiority.
- Building on the SDT, we further develop an offline-to-offline framework that can produce more plausible offline handwritten Chinese characters, as evidenced in Appendix A.4.

2. Related Work

Handwriting generation. Various style-content disentangling methods [1, 8, 18, 21, 24] have been proposed to generate handwritings with arbitrary styles. These methods assume that reference samples can be decomposed into style and content spaces. They disentangle calligraphic styles from reference samples and recombine them with specific textual content for controllable style synthesis. For instance, DeepWriting [1] adopts RNNs to extract the style vectors from online handwritings, and then combines them with character embeddings for synthesizing stylized online handwritings. However, it typically over-smooths the styles of distinct letters and loses key details, since its extracted style vectors are letter-independent [21].

DSD [21] addresses the over-smooth problem by segmenting online handwritten words into isolated letters and encoding them into global and letter-specific style vectors, improving synthetic handwriting quality. Compared with it, our method can explicitly capture more fine-grained details (e.g., stroke length, location and curvature), which are more difficult to obtain. Besides, these methods [1, 21, 34] rely on extra fine annotations to segment input cursive scripts, while our SDT need not. Recently, HWT [3] uses a vanilla Transformer encoder for style pattern extraction. However, these methods [1, 3, 8, 18, 21] rely on complex content references, such as recurrent embeddings and letter-wise fil-

ter maps. SLOGAN [24] addresses this issue by extracting textual contents from easily obtainable printed images, but it struggles with generalizing to unseen handwriting styles due to the fixed writer ID. In contrast, our SDT effectively obtains content and style information, and thus can synthesize characters with arbitrary styles well.

Regarding handwritten Chinese characters, several attempts [4, 16, 20] use GANs to generate Chinese handwriting images, but always result in characteristic artifacts. Drawing [40] and FontRNN [31] adopt RNNs, but they cannot flexibly control the handwriting styles. In addition, DeepImitator [44] and WriteLikeYou [30] propose style-controlled generation using style representations from offline images [44] or online trajectories [30]. Unlike these methods [30, 44] that only extract an overall writer style, our SDT captures both writer and character-level styles, significantly improving the performance of handwriting imitation.

Contrastive learning. Contrastive learning [13, 41] has been widely used in various fields [9, 28, 29, 33, 42, 43]. Some image translation studies [14, 27] use contrastive learning to enhance natural image generation quality, encouraging content preservation during transfer. Unlike these methods [14, 27], our SDT aligns independently sampled patches from the same input, aiming to improve style representations of handwritings.

3. Method

Problem statement. We aim to synthesize stylized online handwritings with conditional content and style. Given a content image I and a set of style images $X_s = \{x_s^i\}_{i=1}^K$, randomly sampled from a writer w_s , we aim to generate an online handwritten Chinese character $\hat{Y}_s$ that reflects the calligraphic style of w_s and retains the textual content of I . The key challenge lies in obtaining discriminative style representations from a limited number of stylized samples.

To address this task, inspired by our observations (cf. Figure 2) of the *overall uniformity* (i.e., writer-wise style) and *inconsistent details* (i.e., character-wise style), we propose to disentangle the style of exemplar writers into overall and detailed styles for enhancing handwriting imitation performance. To achieve this, we introduce a new style-disentangled Transformer (SDT) approach to decouple the two styles from individual handwritings. The overall scheme of SDT is presented below.

3.1. Overall Scheme

As shown in Figure 4, SDT consists of a dual-head style encoder, a content encoder, and a Transformer decoder. The style encoder (cf. Section 3.2) is designed to learn the writer-wise and character-wise styles. It firstly extracts rich calligraphic style patterns from reference style examples X_s via a sequential combination of a CNN and a Transformer

encoder, followed by the writer and glyph heads to disentangle the writer-wise and character-wise styles from the extracted style patterns. To this end, we propose two contrastive objectives, WriterNCE $\mathcal{L}_{wri}$ and GlyphNCE $\mathcal{L}_{gly}$, to encourage the encoder to learn the two styles, respectively. Specifically, $\mathcal{L}_{wri}$ maximizes the mutual information between character instances belonging to the same writer, while $\mathcal{L}_{gly}$ associates the positive pair independently sampled from the same character. Guided by $\mathcal{L}_{wri}$ and $\mathcal{L}_{gly}$, the writer and glyph heads can acquire discriminative writer-wise and character-wise style features, respectively.

The content encoder uses a standard Resnet18 [15] as the CNN backbone to learn the compact feature map $q_{map} \in \mathbb{R}^{h \times w \times c}$ from a content reference I , and feeds the flattened feature patches into a Transformer encoder to extract the textual content representation $q \in \mathbb{R}^{d \times c}$, where $d = h \times w$ and c is the channel dimension. Benefiting from the strong capability of Transformer to capture long-range dependencies between feature patches, the content encoder expects an informative content feature q with a global context.

After the two encoders, a multi-layer Transformer decoder (cf. Section 3.3) is used to synthesize $\hat{Y}_s$ in an autoregressive fashion, conditioned on the two style representations and the content feature. This decoder is supervised by the pen moving prediction loss $\mathcal{L}_{pre}$ and pen state classification loss $\mathcal{L}_{cls}$.

To summarize, the overall training objective of SDT combines all four loss functions:

$$\mathcal{L} = \mathcal{L}_{wri} + \mathcal{L}_{gly} + \mathcal{L}_{pre} + \lambda \mathcal{L}_{cls}, \quad (1)$$

where λ serves as a trade-off factor. Each component of our SDT is detailed in the following Section 3.2 and Section 3.3.

3.2. Dual-head Style Encoder

As illustrated in Figure 2, there are two distinct styles in a person’s handwriting: writer-wise and character-wise styles. We propose a dual-head style encoder to obtain the two style representations. As shown in Figure 4, the input $X = \{x^i\}_{i=1}^K$ is firstly encoded by ResNet18 to obtain a sequence of feature maps $F_m = \{f_m^i\}_{i=1}^K \in \mathbb{R}^{K \times h \times w \times c}$. Next, we flatten the spatial dimension of each feature map to obtain feature sequences $F = \{f^i\}_{i=1}^K \in \mathbb{R}^{K \times d \times c}$. These feature sequences are then fed into a Transformer encoder to extract more informative feature sequences $Z = \{z^i\}_{i=1}^K \in \mathbb{R}^{K \times d \times c}$. Finally, we use the two heads, built on self-attention [35], to further disentangle Z into the writer-wise style representations $E = \{e^i\}_{i=1}^K \in \mathbb{R}^{K \times d \times c}$ via $\mathcal{L}_{wri}$ and the character-wise counterparts $G = \{g^i\}_{i=1}^K \in \mathbb{R}^{K \times d \times c}$ via $\mathcal{L}_{gly}$, respectively. We next detail the two contrastive learning objectives as follows.

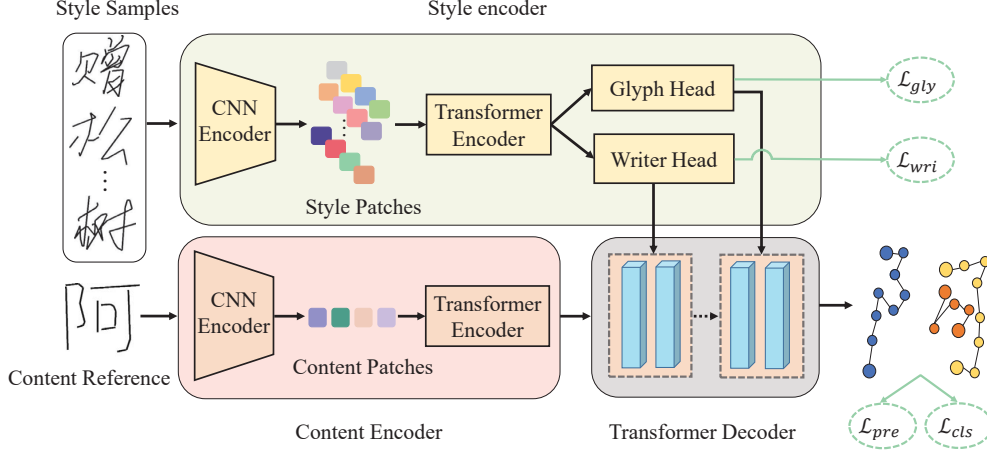


Figure 4. Overview of the proposed method. Our SDT consists of a dual-head style encoder, a content encoder and a Transformer decoder. The writer-wise and character-wise style representations extracted by style encoder and the learned content feature are fed into the Transformer decoder to progressively generate online handwritings. We utilize the WriterNCE $\mathcal{L}_{wri}$ and GlyphNCE $\mathcal{L}_{gly}$ to guide the two heads for learning the corresponding styles. $\mathcal{L}_{pre}$ and $\mathcal{L}_{cls}$ denote the pen moving prediction and state classification loss, respectively.

3.2.1 Writer-wise Contrastive Learning

To explicitly encourage the writer head to learn the writer-wise style, we propose to learn a feature space where the style features from the same writer are closer than those from different writers. The intuition is that one person’s handwritings consistently exhibit similar style information, which can serve as a crucial clue for distinguishing writers. To this end, we take characters written by the same person as positive pairs and those from different writers as negative samples, and develop a new WriterNCE loss for writer-wise style learning.

Specifically, let $j \in M = \{1, \dots, N\}$ be the index of an element within a mini-batch and let $A(j) = M \setminus \{j\}$ be other indices distinct from j , where N is the batch size. Given a writer-wise style feature e_j belonging to writer w_j as the anchor, we denote its in-batch positive sample set as $P(j) = \{p \in A(j) : w_p = w_j\}$ and its negative set as $A(j) \setminus P(j)$. The WriterNCE loss is formulated as follows:

$$\mathcal{L}_{wri} = \frac{-1}{N} \sum_{j \in M} \frac{1}{|P(j)|} \sum_{p \in P(j)} \log \frac{\exp(\text{sim}(e_j, e_p)/\tau)}{\sum_{a \in A(j)} \exp(\text{sim}(e_j, e_a)/\tau)}, \quad (2)$$

where $\text{sim}(e_j, e_p) = f_1(e_j)^\top f_1(e_p)$, τ is a temperature parameter and $f_1(\cdot)$ is a multi-layer perceptron (MLP) that projects features to a ℓ_2 -normalized feature space where $\mathcal{L}_{wri}$ is applied.

3.2.2 Character-wise Contrastive Learning

Compared with the overall writer-wise style, character-wise style differences often exist in the fine details of distinct characters, e.g., stroke length and curvature (cf. Figure 3). Inspired by this, we aim to maximize the mutual informa-

tion between diverse views of a character, thereby enforcing the glyph head to learn the character-wise style. As strokes are distributed across arbitrary spatial locations in character images, we propose to capture stroke details via contrastive learning, by randomly selecting a small subset of patches following a uniform distribution. Specifically, we conduct sampling over the sequential patch tokens obtained from the glyph head.

Given character-wise style features $\{g_j\}_{j=1}^B \in \mathbb{R}^{B \times d \times c}$ extracted from B characters, we sample a positive patch pair (i.e., $o \in \mathbb{R}^{n \times c}$ and $o^+ \in \mathbb{R}^{n \times c}$) from the same randomly selected g , and $B-1$ negative patches $\{o_j^-\}_{j=1}^{B-1}$ from the remaining $B-1$ style features. Here, the number of sampled patch tokens is $n = d \cdot \alpha$, where α is the sampling ratio. The GlyphNCE loss is formulated as:

$$\mathcal{L}_{gly} = -\log \frac{\exp(\text{sim}(o, o^+)/\tau)}{\exp(\text{sim}(o, o^+)/\tau) + \sum_{j=1}^{B-1} \exp(\text{sim}(o, o_j^-)/\tau)}, \quad (3)$$

where $\text{sim}(o, o^+) = f_2(o)^\top f_2(o^+)$, and $f_2(\cdot)$ is an MLP with the same structure as $f_1(\cdot)$.

3.3. Transformer Decoder for Handwriting

The goal of the Transformer decoder is to progressively generate realistic online characters, denoted as $\hat{Y}$, based on the global content feature q and the obtained style representations (i.e., $E = \{e^i\}_{i=1}^K$ and $G = \{g^i\}_{i=1}^K$). As each online character consists of numerous points (i.e., $\hat{Y} = \{\hat{y}_j\}_{j=1}^L$, with L being the total number of points; see more details in Appendix A.10), the decoder faces the challenge of effectively integrating content and style features to accurately depict all points of the character. To address this challenge, we propose to fuse q , E and H within the multi-head attention layers of our Transformer decoder. As shown

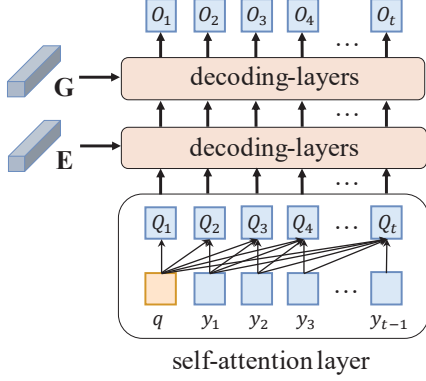


Figure 5. Illustration of the fusion of the style information in our Transformer decoder. At each time step, the query vector is first encoded from the content feature q and previous points $\{y_j\}_{j=1}^{t-1}$. Then, it successively attends to the writer-wise and character-wise style features (*i.e.*, E and G) for adaptively aggregating style information, which is finally decoded into the current-step output.

in Figure 5, E and H serve as the key/value vectors, while Q serves as the query vector that successively attends to E and G to aggregate style information.

During training, at any decoding step t , we take q as the initial point. As shown in Figure 5, we apply self-attention to the point sequence, *i.e.* the content context $[q, y_1, \dots, y_{t-1}]$, to obtain the query vector $Q_t \in \mathbb{R}^c$. Here, the ground-truth points $\{y_j\}_{j=1}^{t-1}$ are used to accelerate model convergence in training [30, 37]. Subsequently, Q_t attends to E and G over the subsequent decoding layers to adaptively aggregate style information, ultimately generating the output $O_t \in \mathbb{R}^{6R+3}$. The output with $6R+3$ logits is then used to generate the pen moving $(\Delta \hat{u}_t, \Delta \hat{v}_t)$ and the pen state $(\hat{m}_t^1, \hat{m}_t^2, \hat{m}_t^3)$. Specifically, we use a Gaussian mixture model (GMM) [11] with R bivariate normal distributions to predict the pen moving, with each normal distribution containing 6 parameters. Moreover, we use the other 3 logits to generate the pen state. The training loss for supervising the decoder comprises two parts: the pen movement prediction loss $\mathcal{L}_{pre}(\Delta \hat{u}_t, \Delta \hat{v}_t)$, and the pen state classification loss $\mathcal{L}_{cls}(\hat{m}_t^1, \hat{m}_t^2, \hat{m}_t^3)$. Please refer to Appendix A.11 for more details of these losses.

The inference phase is different from the training phase, where the ground truth y is not available at test time. Instead, we take the generated points $\{\hat{y}_j\}_{j=1}^{t-1}$ as the input of the step t , and combine them with q , E , and G to predict the next point $\hat{y}_t$. Such a process repeats until a pen-end state $(\hat{m}_{t-1}^3=1)$ is received.

4. Experiments

4.1. Chinese handwriting generation

Dataset. To evaluate SDT in generating Chinese handwritings, we use CASIA-OLHWDB (1.0-1.2) [22] for model

training and ICDAR-2013 competition database [38] for testing. The training set consists of about 3.7 million online Chinese handwritten characters produced by 1,020 writers, while the test set contains 60 writers, with each contributing 3,755 most frequently used characters set of GB2312-80. Following [12], we use the Ramer–Douglas–Peucker algorithm [7] with a parameter of $\epsilon = 2$ to remove redundant points of characters, leading to an average sequence length of 50. Following [44], we render offline style references using coordinate points of online characters, and each style sample is randomly sampled from the target-writer handwritings during inference, as shown in Figure 4. For content images, we use the popular average Chinese font [17].

Evaluation metrics. We use Dynamic Time Warping (DTW) [2, 5], an elastic matching technique for aligning the given two sequences, to calculate the distance between the generated and real characters. Moreover, we use Content Score [44] to measure the structure correctness of generated characters, and adopt Style Score [30] to quantify the style similarity between the generated and real handwritings. We also conduct user preference studies to quantify the subjective quality of the output characters. More details are provided in Appendix A.1.1.

Implementation details. We set the number of the style reference to $K = 15$, and resize the reference style and content images to 64×64 . Moreover, each Transformer encoder contains 2 self-attention layers, while the Transformer decoder has 4 layers for obtaining style representations (2 for writer-wise and 2 for character-wise). Following the original Transformer [35], each Transformer layer contains the multi-head attention with $c = 512$ dimensional states and 8 attention heads. Contrary to HWT [3], where F is concatenated before the Transformer encoder, we process each feature sequence $f \in F$ individually. Moreover, we apply sinusoidal positional encoding [35] to input tokens before feeding them to the Transformer encoder and decoder. For training, we first pre-train the content encoder with 138k iterations (batch size of 256) for character classification on training samples and then train the whole model with 148k iterations (batch size of 128), on a single RTX3090 GPU. The optimizer is Adam [19], with a learning rate of 0.0002 and gradient clipping of 5.0. The sampling ratio α is determined through a search over 0.25, 0.5, 0.75, 1.00, and 0.25 is chosen. Following [30], we set $\lambda = 2$. Further implementation details are provided in Appendix A.1.2.

Compared methods. We compare our proposed SDT with state-of-the-art online Chinese character generation methods, *i.e.* Drawing [40], FontRNN [31], DeepImitator [44], and WriteLikeYou [30]. To ensure a fair comparison, we re-implement Drawing and FontRNN by adding the style branch of DeepImitator [44], enabling them to achieve arbitrary stylized character generation. To adapt WriteLikeYou [30] to handle images, we update its encoder to

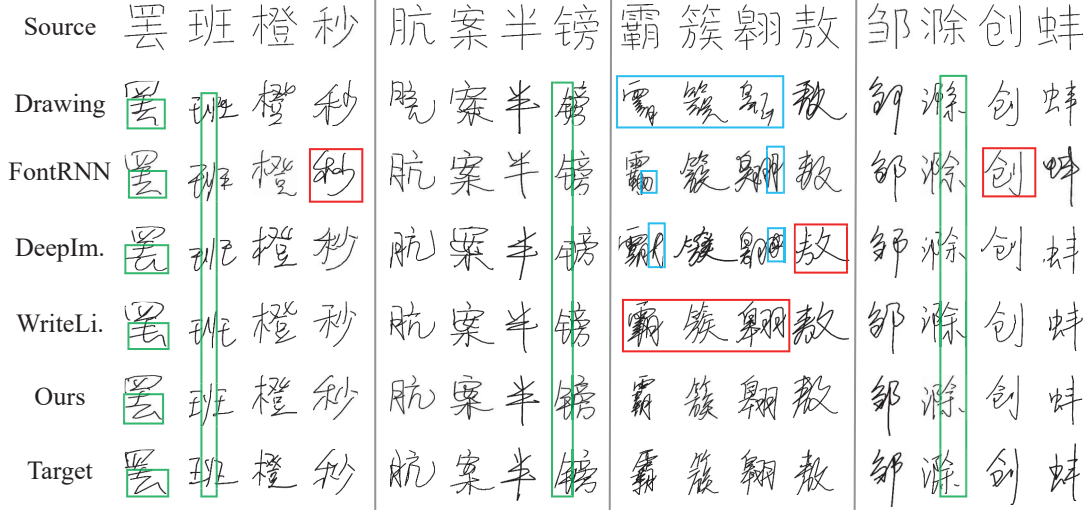


Figure 6. Comparisons with the state-of-the-art methods for online Chinese handwriting generation. The red and blue boxes highlight failures of style imitation and structure preservation, respectively. WriteLi. indicates the WriteLikeYou-v2. The green boxes highlight comparisons between the fine details of targets and generated characters.

Table 1. Comparisons with state-of-the-art methods on Chinese dataset. See more results of WriteLikeYou [30] in Appendix A.6.

Method	Style Score $\uparrow$	Content Score $\uparrow$	DTW $\downarrow$	User Prefer. (%) $\uparrow$
Drawing [40]	35.83	78.15	1.1813	3.53
FontRNN [31]	46.14	92.18	1.0448	7.07
DeepImitator [44]	50.67	90.92	1.0622	7.99
WriteLikeYou-v1 [30]	71.09	93.89	0.9832	11.67
WriteLikeYou-v2 [30]	72.37	96.44	0.9289	13.07
SDT(Ours)	94.50	97.04	0.8789	56.67

create two new variants: WriteLikeYou-v1 (*i.e.*, CNN encoder [44]) and WriteLikeYou-v2 (*i.e.*, the same CNN-Transformer encoder as our SDT), based on the released official code¹. More details are available in Appendix A.1.3.

Quantitative comparison. Quantitative results are in Table 1, revealing that SDT outperforms other methods across all evaluation metrics. Notably, SDT surpasses the second-best method by a significant 22.13% margin in Style Score, demonstrating the proposed method’s exceptional style imitation performance.

Qualitative comparison. We visualize the generated samples of each method in Figure 6, which intuitively explains the significant superiority of SDT in the user preference study. In Figure 6, Drawing [40] generates the least satisfactory results, often producing unreadable characters. FontRNN [31] and DeepImitator [44] occasionally synthesize unpleasant stroke paddings, and WriteLikeYou [30] struggles with generating complex characters regarding style mimicry. In contrast, our method yields higher-quality results, particularly in recovering fine character details.

¹<https://github.com/ShusenTang/WriteLikeYou>

Table 2. Ablation study. Here, we further use FID to measure the distance between generated and real samples for each writer separately, and finally average them.

writer-wise	character-wise	Generated Samples	Style Score $\uparrow$	FID $\downarrow$	DTW $\downarrow$
		倾 解 朝	85.52	27.75	0.8941
✓		倾 解 朝	91.38	26.38	0.8841
	✓	倾 解 朝	90.31	26.89	0.8803
✓	✓	倾 解 朝	94.50	25.46	0.8789
Ground Truth		倾 解 朝			

4.2. Analysis

In this section, we assess the impact of the two learned style features and that of the style inputs. We also analyze the effect of the sampling ratio α , and discuss the combination strategies in the Transformer decoder in Appendix A.5. Besides, we discuss failure cases in Appendix A.7.

Quantitative evaluation of two style representations. We ablate the effects of the two extracted style representations in Table 2. The findings include: (1) Both style representations enhance the quality of generated outcomes in all evaluation metrics, particularly in terms of Style Score, improving by 4.79% and 5.86%, respectively. (2) Combining the two style features further boosts model performance across all evaluation metrics, suggesting that the extracted styles are complementary. Moreover, the order in which the two style features are input into the Transformer decoder has no significant impact on Style Score (93.72% vs. 94.50%).

Qualitative comparison between two style representations. To further analyze the differences between the

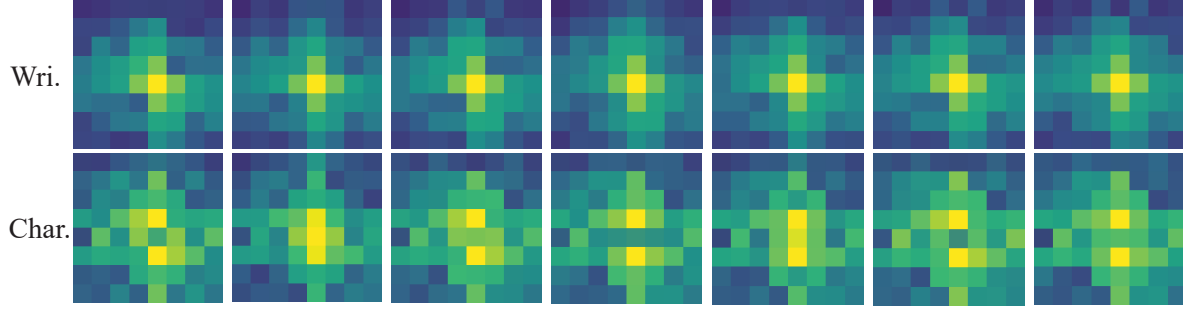


Figure 7. Spectrum analysis of two style representations. We provide frequency magnitude visualizations belonging to 7 writers, where the top row shows the writer-wise style while the bottom represents the character-wise one. Each spectrum map is averaged over 100 character samples written by the same person. The brighter the color, the larger the magnitude. A pixel farther from the center means a higher frequency [26]. We find that the writer-wise style representations capture more low-frequency information, while character-wise style representations capture more high-frequency information.

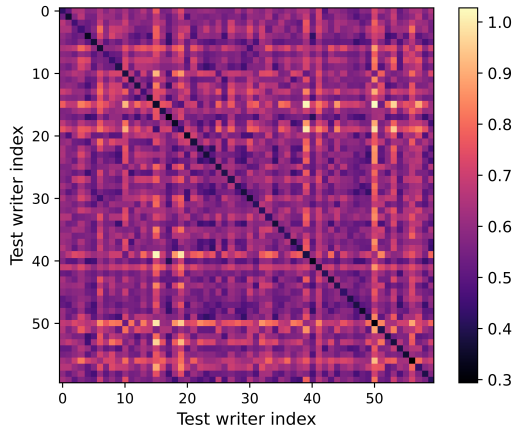


Figure 8. The heat map of the DTW matrix. The dark diagonal indicates that the generated characters still own a higher similarity even using different X_s belonging to the same writer.

two styles, we resize the output patch tokens representing the two styles to feature maps, respectively, and visualize their frequency magnitudes in Figure 7. We find that character-wise style features capture more high-frequency information, whereas writer-wise features mainly focus on low-frequency information. According to [6], the high-frequency information in an image usually captures fine details while the low frequencies contain the overall part of objects. This finding supports our motivation that the writer head helps to imitate the overall style (*e.g.*, glyph slant), while the glyph head captures the detailed style (*e.g.*, stroke curvature), as shown in Table 2.

Effect of using different style inputs. As mentioned in [30], given different style inputs X_s belonging to the same writer w_s , the imitation model may generate inconsistent characters. To evaluate the effect of different style inputs, we conduct two independent experiments using different X_s based on the same model. Following [30], we use the model to generate 200 characters for each writer in the test set. We then calculate the DTW distance between corre-

Table 3. Quantitative evaluations of our SDT and competitors on Japanese, Indic, and English datasets.

Datasets	Methods	Content Score $\uparrow$	DTW $\downarrow$
Japanese	Drawing [40]	50.74	1.4657
	DeepImitator [44]	53.20	1.2564
	WriteLikeYou-v2 [30]	85.61	1.2066
	SDT(Ours)	91.31	1.1289
Indic	Drawing [40]	2.34	9.8230
	DeepImitator [44]	4.13	6.7421
	WriteLikeYou-v2 [30]	13.19	4.5130
	SDT(Ours)	97.22	0.7075
English	Drawing [40]	79.14	1.8519
	DeepImitator [44]	76.53	1.6460
	WriteLikeYou-v2 [30]	84.41	1.6215
	SDT(Ours)	85.52	1.6048

sponding characters individually and average them according to the writer index (see Appendix A.1.4 for more details) to get a DTW square matrix. The resulting DTW square matrix is visualized in Figure 8. The dark diagonal in Figure 8 suggests that the generated characters maintain a high degree of similarity even when using different X_s belonging to the same writer, demonstrating that our SDT can generate consistent results from various style inputs.

4.3. Applications to Other Languages

Japanese handwriting generation. For the Japanese handwriting generation task, we conduct experiments on TUAT HANDS [25] database (see more details in Appendix A.1.5) to evaluate the effectiveness of our method. Figure 9 (a) and Table 3 verify the effectiveness of SDT for Japanese handwriting generation. Specifically, from Table 3, we observe that our SDT outperforms all compared methods in terms of two quantitative metrics, indicating that SDT performs well in multiple languages. Furthermore, we provide additional evaluation metrics in Appendix A.8.

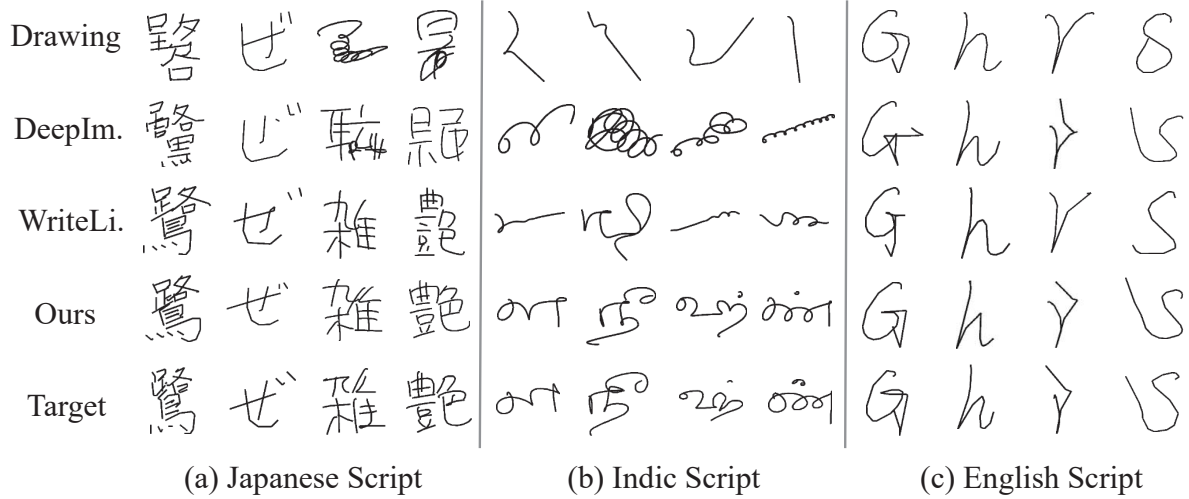


Figure 9. Comparisons with the competitors for online handwriting generation on various scripts. WriteLi. indicates WriteLikeYou-v2.

Indic handwriting generation. We evaluate our method on the Indic handwriting generation task based on the Tamil² dataset. It is worth noting that Indic handwriting generation presents a more significant challenge, as Indic characters contain more trajectory points than Chinese, Japanese, and English scripts (i.e., 88 vs. 68, 50, 30 on average; see Appendix A.1.5 for more dataset information). We compare our method with other approaches on the official test set in terms of Content Score and Dynamic Time Warping (DTW). As shown in Table 3, we find that our SDT significantly outperforms the second-best method in terms of the two quantitative metrics, achieving an 84.03% higher Content Score and a 3.8055 lower DTW. This indicates that our SDT can handle handwritten characters with a large number of points (averaging 88) and ensure the quality of synthetic samples, as illustrated in Figure 9 (b). The potential advantages of our SDT are: (1) The improved style representations extracted by our SDT prevent the collapse of generated characters. (2) Our Transformer decoder facilitates long-distance dependence between trajectory points. We provide more experimental analysis in Appendix A.9.

English handwriting generation. To evaluate our method in generating English handwritings, we collect all of the English samples from the symbol part of CASIA-OLHWDB(1.0-1.2) [22] and ICDAR-2013 competition database [38] (see more details in Appendix A.1.5). Similarly, we use Content Score and DTW as evaluation metrics. As shown in Figure 9 (c), we find that all methods achieve sound and comparable performance. One reason for this is that the English script contains fewer character classes and a smaller number of trajectory points (averaging 30), making

their imitation easier compared to other scripts. Nevertheless, our SDT still outperforms other methods by a small margin in terms of both Content Score and DTW, as shown in Table 3. Moreover, we observe that corresponding uppercase and lowercase letters sometimes exhibit subtle inter-class differences (e.g., O vs. o), which leads our SDT to achieve a relatively low Content Score.

5. Conclusion

In this paper, we have proposed a novel method, named style-disentangled Transformer (SDT), to synthesize realistic and diverse online handwritings. SDT enhances imitation performance by disentangling the writer-wise and character-wise style representations from individual handwriting samples. For the writer-wise style, we group characters from the same writer and separate those from different writers, promoting SDT’s ability to learn uniformity in individual handwritings. For the character-wise style, we maximize the mutual information between the distinct views of a character. Moreover, we extend SDT and introduce an offline-to-offline framework for improving the generation quality of offline Chinese handwritings. Promising results on various language scripts verify the effectiveness of our SDT. Although primarily designed for handwriting generation, SDT still holds the potential for extension to other generative tasks, such as font generation.

Acknowledgments This research is partially supported by NSFC (Grant No.62176093, 61673182), Key Realm R&D Program of Guangzhou (No.202206030001), Guangdong Basic and Applied Basic Research Foundation (No.2021A1515012282) and Guangdong Provincial Key Laboratory of Human Digital Twin (2022B1212010004).

²<http://lipitk.sourceforge.net/datasets/tamilchardata.htm>

References

- [1] Emre Aksan, Fabrizio Pece, and Otmar Hilliges. Deepwriting: Making digital ink editable via deep generative modeling. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–14, 2018. 2
- [2] Donald J Berndt and James Clifford. Using dynamic time warping to find patterns in time series. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining Workshop*, 1994. 5
- [3] Ankan Kumar Bhunia, Salman Khan, Hisham Cholakkal, Rao Muhammad Anwer, Fahad Shahbaz Khan, and Mubarak Shah. Handwriting transformers. In *International Conference on Computer Vision*, pages 1086–1094, 2021. 2, 5
- [4] Bo Chang, Qiong Zhang, Shenyi Pan, and Lili Meng. Generating handwritten chinese characters using cyclegan. In *Winter Conference on Applications of Computer Vision*, pages 199–207, 2018. 3
- [5] Zhounan Chen, Daihui Yang, Jinglin Liang, Xinwu Liu, Yuyi Wang, Zhenghua Peng, and Shuangping Huang. Complex handwriting trajectory recovery: Evaluation metrics and algorithm. In *Asian Conference on Computer Vision*, pages 1060–1076, 2022. 5
- [6] James W Cooley, Peter AW Lewis, and Peter D Welch. The fast fourier transform and its applications. *IEEE Transactions on Education*, 12(1):27–34, 1969. 7
- [7] David H Douglas and Thomas K Peucker. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica: the international journal for geographic information and geovisualization*, pages 112–122, 1973. 5
- [8] Ji Gan and Weiqiang Wang. Higan: Handwriting imitation conditioned on arbitrary-length texts and disentangled styles. In *AAAI Conference on Artificial Intelligence*, pages 7484–7492, 2021. 2
- [9] Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings. In *Empirical Methods in Natural Language Processing*, pages 6894–6910, 2021. 3
- [10] Yue Gao, Yuan Guo, Zhouhui Lian, Yingmin Tang, and Jianguo Xiao. Artistic glyph image synthesis via one-stage few-shot learning. *ACM Transactions on Graphics*, 38(6):1–12, 2019. 1
- [11] Alex Graves. Generating sequences with recurrent neural networks. *Arxiv*, 2013. 5
- [12] David Ha and Douglas Eck. A neural representation of sketch drawings. In *International Conference on Learning Representations*, 2018. 5
- [13] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *Computer Vision and Pattern Recognition*, pages 1735–1742, 2006. 2, 3
- [14] Junlin Han, Mehrdad Shoeiby, Lars Petersson, and Mohammad Ali Armin. Dual contrastive learning for unsupervised image-to-image translation. In *Computer Vision and Pattern Recognition*, pages 746–755, 2021. 3
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 3
- [16] Hongxiang Huang, Daihui Yang, Gang Dai, Zhen Han, Yuyi Wang, Kin-Man Lam, Fan Yang, Shuangping Huang, Yongge Liu, and Mengchao He. Agtgan: Unpaired image translation for photographic ancient character generation. In *ACM International Conference on Multimedia*, pages 5456–5467, 2022. 3
- [17] Yue Jiang, Zhouhui Lian, Yingmin Tang, and Jianguo Xiao. Sfont: Structure-guided chinese font generation via deep stacked networks. In *AAAI conference on Artificial Intelligence*, pages 4015–4022, 2019. 5
- [18] Lei Kang, Pau Riba, Yaxing Wang, Marçal Rusinol, Alicia Fornés, and Mauricio Villegas. Ganwriting: content-conditioned generation of styled handwritten word images. In *European Conference on Computer Vision*, pages 273–289, 2020. 1, 2
- [19] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. 5
- [20] Weirui Kong and Bicheng Xu. Handwritten chinese character generation via conditional neural generative models. In *Advances in Neural Information Processing Systems*, pages 4–7, 2017. 3
- [21] Atsunobu Kotani, Stefanie Tellex, and James Tompkin. Generating handwriting via decoupled style descriptors. In *European Conference on Computer Vision*, pages 764–780, 2020. 2
- [22] Cheng-Lin Liu, Fei Yin, Da-Han Wang, and Qiu-Feng Wang. Casia online and offline chinese handwriting databases. In *International Conference on Document Analysis and Recognition*, pages 37–41, 2011. 5, 8
- [23] Wei Liu, Fangyue Liu, Fei Ding, Qian He, and Zili Yi. Xmpfont: Self-supervised cross-modality pre-training for few-shot font generation. In *Computer Vision and Pattern Recognition*, pages 7905–7914, 2022. 1
- [24] Canjie Luo, Yuanzhi Zhu, Lianwen Jin, Zhe Li, and Dezhi Peng. Slogan: Handwriting style synthesis for arbitrary-length and out-of-vocabulary text. *IEEE Transactions on Neural Networks and Learning Systems*, 2022. 2, 3
- [25] Kaoru Matsumoto, Takahiro Fukushima, and Masaki Nakagawa. Collection and analysis of on-line handwritten japanese character patterns. In *International Conference on Document Analysis and Recognition*, pages 496–500, 2001. 7
- [26] Zizheng Pan, Jianfei Cai, and Bohan Zhuang. Fast vision transformers with hilo attention. In *NeurIPS*, 2022. 7
- [27] Taesung Park, Alexei A Efros, Richard Zhang, and Jun-Yan Zhu. Contrastive learning for unpaired image-to-image translation. In *European Conference on Computer Vision*, pages 319–345, 2020. 3
- [28] Zhen Qiu, Yifan Zhang, Hongbin Lin, Shuaicheng Niu, Yanxia Liu, Qing Du, and Mingkui Tan. Source-free domain adaptation via avatar prototype generation and adaptation. In *International Joint Conference on Artificial Intelligence*, 2021. 3

- [29] Xuanchi Ren, Tao Yang, Yuwang Wang, and Wenjun Zeng. Learning disentangled representation by exploiting pretrained generative models: A contrastive learning view. In *International Conference on Learning Representations*, 2022. 3
- [30] Shusen Tang and Zhouhui Lian. Write like you: Synthesizing your cursive online chinese handwriting via metric-based meta learning. *Computer Graphics Forum*, 40(2):141–151, 2021. 1, 3, 5, 6, 7
- [31] Shusen Tang, Zeqing Xia, Zhouhui Lian, Yingmin Tang, and Jianguo Xiao. Fontrnn: Generating large-scale chinese fonts via recurrent neural network. *Computer Graphics Forum*, 38(7):567–577, 2019. 3, 5, 6
- [32] Yuchen Tian. zi2zi:master chinese calligraphy with conditional adversarial networks. <https://github.com/kaonashi-tyc/zi2zi>, 2017. 1
- [33] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive representation distillation. In *International Conference on Learning Representations*, 2020. 3
- [34] Ruben Tolosana, Paula Delgado-Santos, Andres Perez-Urbe, Ruben Vera-Rodriguez, Julian Fierrez, and Aythami Morales. Deepwritsyn: On-line handwriting synthesis via deep short-term representations. In *AAAI Conference on Artificial Intelligence*, pages 600–608, 2021. 2
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017. 2, 3, 5
- [36] Hao Wang, Yitong Wang, Zheng Zhou, Xing Ji, Dihong Gong, Jingchao Zhou, Zhifeng Li, and Wei Liu. Cosface: Large margin cosine loss for deep face recognition. In *Computer Vision and Pattern Recognition*, pages 5265–5274, 2018. 1
- [37] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989. 5
- [38] Fei Yin, Qiu-Feng Wang, Xu-Yao Zhang, and Cheng-Lin Liu. Icdar 2013 chinese handwriting recognition competition. In *International Conference on Document Analysis and Recognition*, pages 1464–1470, 2013. 5, 8
- [39] Hang Yin, Patrícia Alves-Oliveira, Francisco S Melo, Aude Billard, and Ana Paiva. Synthesizing robotic handwriting motion by learning from human demonstrations. In *International Joint Conference on Artificial Intelligence*, pages 3530–3537, 2016. 1
- [40] Xu-Yao Zhang, Fei Yin, Yan-Ming Zhang, Cheng-Lin Liu, and Yoshua Bengio. Drawing and recognizing chinese characters with recurrent neural network. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):849–862, 2017. 1, 3, 5, 6, 7
- [41] Yifan Zhang, Bryan Hooi, Dapeng Hu, Jian Liang, and Jiashi Feng. Unleashing the power of contrastive self-supervised visual models via contrast-regularized fine-tuning. *Advances in Neural Information Processing Systems*, 34:29848–29860, 2021. 3
- [42] Yifan Zhang, Bryan Hooi, HONG Lanqing, and Jiashi Feng. Self-supervised aggregation of diverse experts for test-agnostic long-tailed recognition. In *Advances in Neural Information Processing Systems*, 2022. 3
- [43] Yifan Zhang, Bingyi Kang, Bryan Hooi, Shuicheng Yan, and Jiashi Feng. Deep long-tailed learning: A survey. *arXiv preprint arXiv:2110.04596*, 2021. 3
- [44] Bocheng Zhao, Jianhua Tao, Minghao Yang, Zhengkun Tian, Cunhang Fan, and Ye Bai. Deep imitator: Handwriting calligraphy imitation via deep attention networks. *Pattern Recognition*, 104:107080, 2020. 1, 3, 5, 6, 7